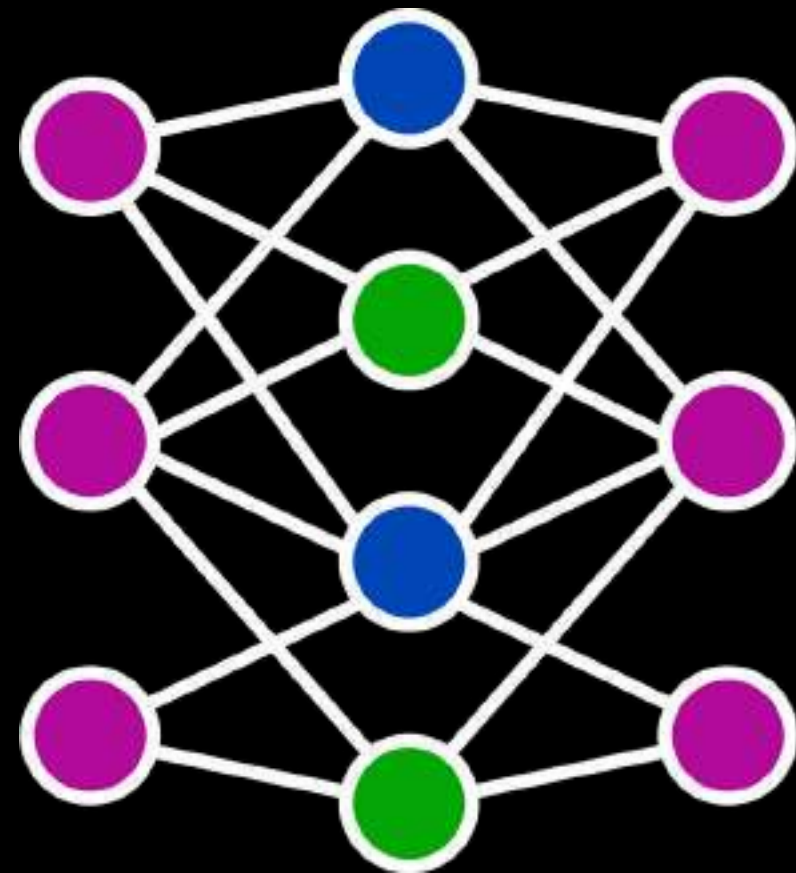


Summer Lectures 2026



Lecture 2

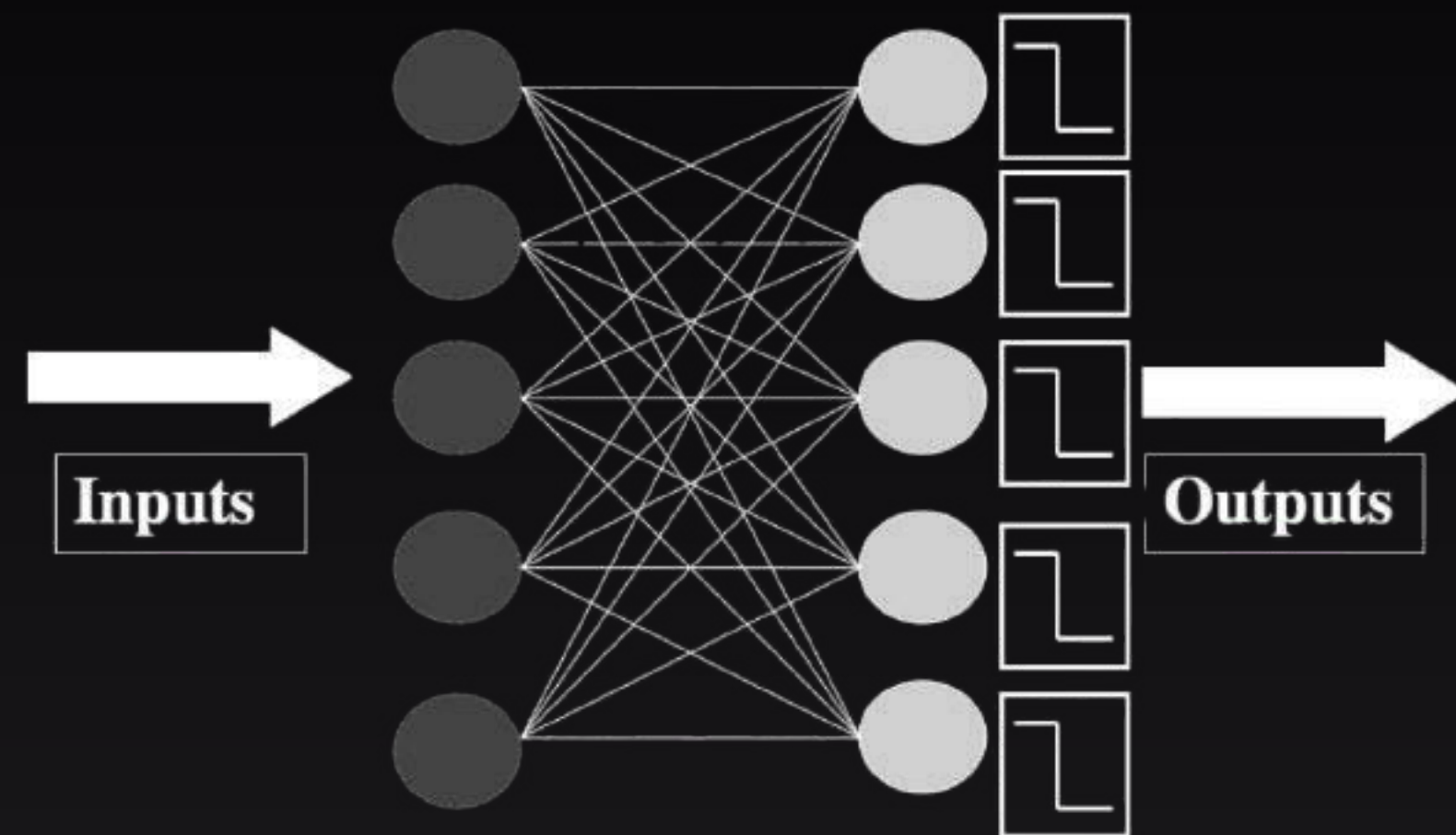
The perceptron Network

- Neurons are completely independent each other

- Notation w_{ij}

Where i runs over the number of neurons and j the number of inputs

- What happens with weights?



How to update weights? A first approach

$$w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j) \cdot x_i$$

LEARNING RATE

How fast the network learns

PREDICTION

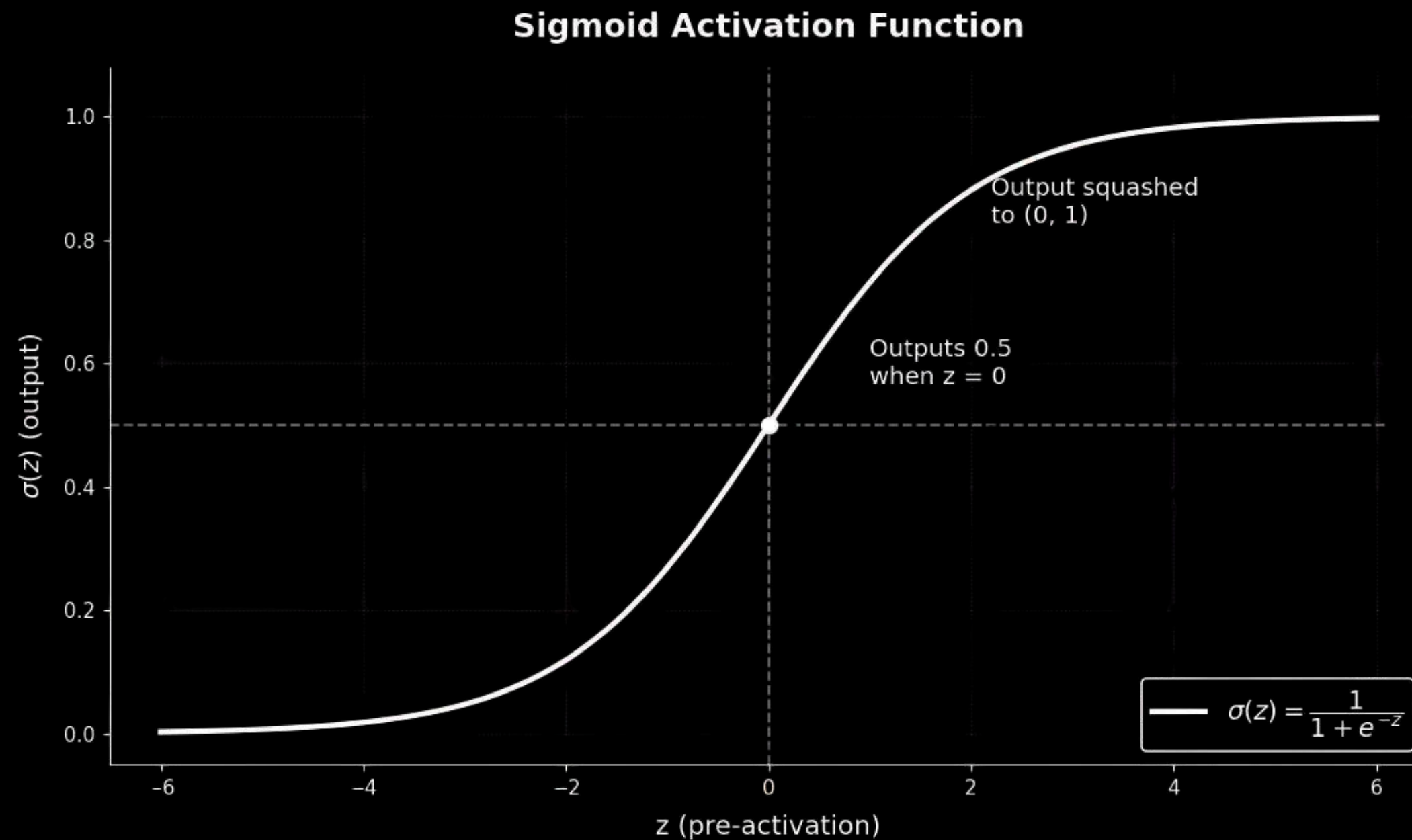
TARGET

INPUT

In order to improve the network
needs to be shown every
training example several times

EPOCHS

The sigmoid

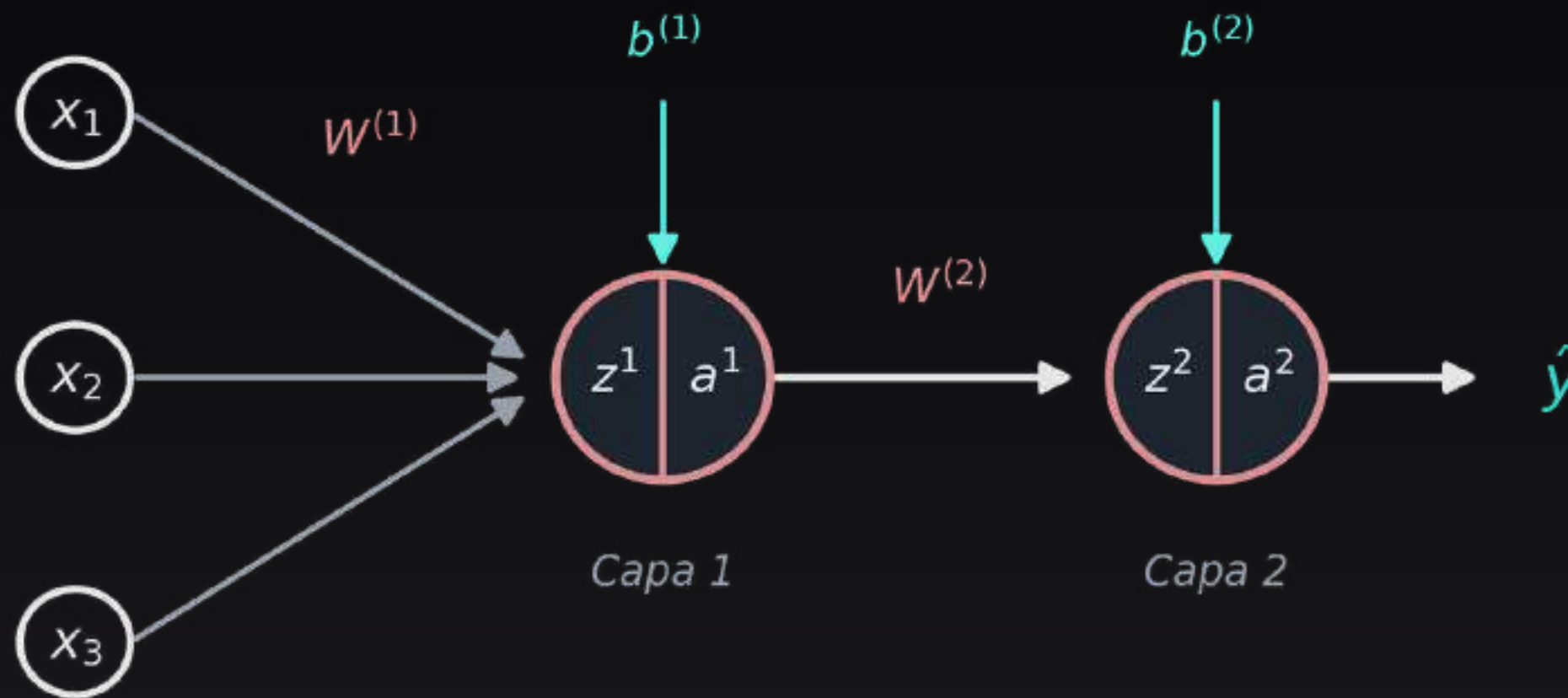




“Without nonlinearity, two
layers are equivalent to a single
layer”

Layers

Stacking multiple linear layers collapses the entire network into a single linear transformation.



Layer 1

$$a^{(1)} = W^{(1)} x + b^{(1)}$$

Layer 2

$$a^{(2)} = W^{(2)} a^{(1)} + b^{(2)}$$

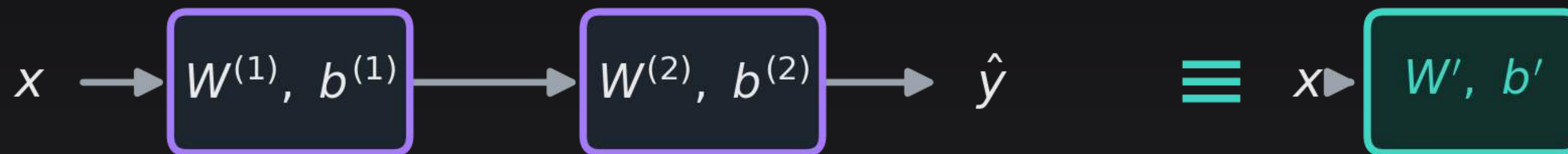
Output

$$\hat{y} = a^{(2)}$$

More in detail

A single weight matrix W' and a single bias b'

$$\begin{aligned} a^{(2)} &= W^{(2)} (W^{(1)}x + b^{(1)}) + b^{(2)} \\ &= (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)}) \\ &= W'x + b' \end{aligned}$$



The Perceptron Algorithm

- **Initialisation**

- set all of the weights w_{ij} to small (positive and negative) random numbers

- **Training**

- for T iterations or until all the outputs are correct:

- * for each input vector:

- compute the activation of each neuron j using activation function g

$$y_j = g \left(\sum_{i=0}^m w_{ij} x_i \right) = \begin{cases} 1 & \text{if } \sum_{i=0}^m w_{ij} x_i > 0 \\ 0 & \text{if } \sum_{i=0}^m w_{ij} x_i \leq 0 \end{cases}$$

- update each of the weights individually using:

$$w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j) \cdot x_i$$

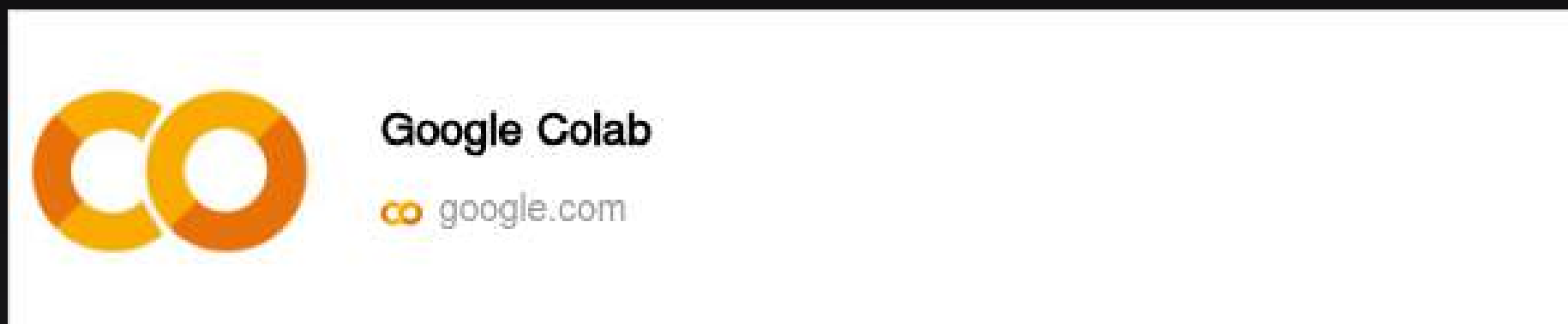
- **Recall**

- compute the activation of each neuron j using:

$$y_j = g \left(\sum_{i=0}^m w_{ij} x_i \right) = \begin{cases} 1 & \text{if } w_{ij} x_i > 0 \\ 0 & \text{if } w_{ij} x_i \leq 0 \end{cases}$$

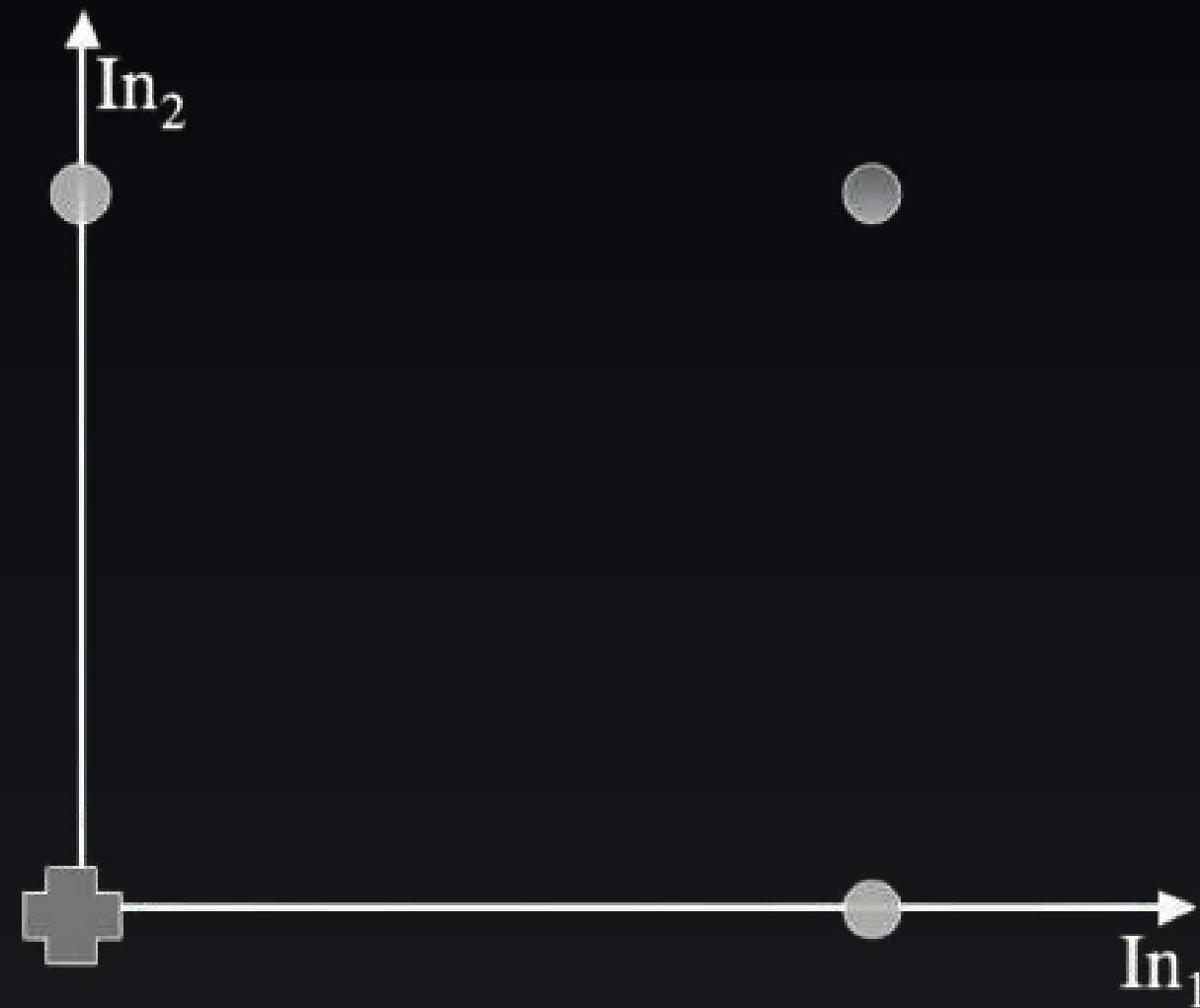
Time to code

1st Proyecto



Logic Functions: OR

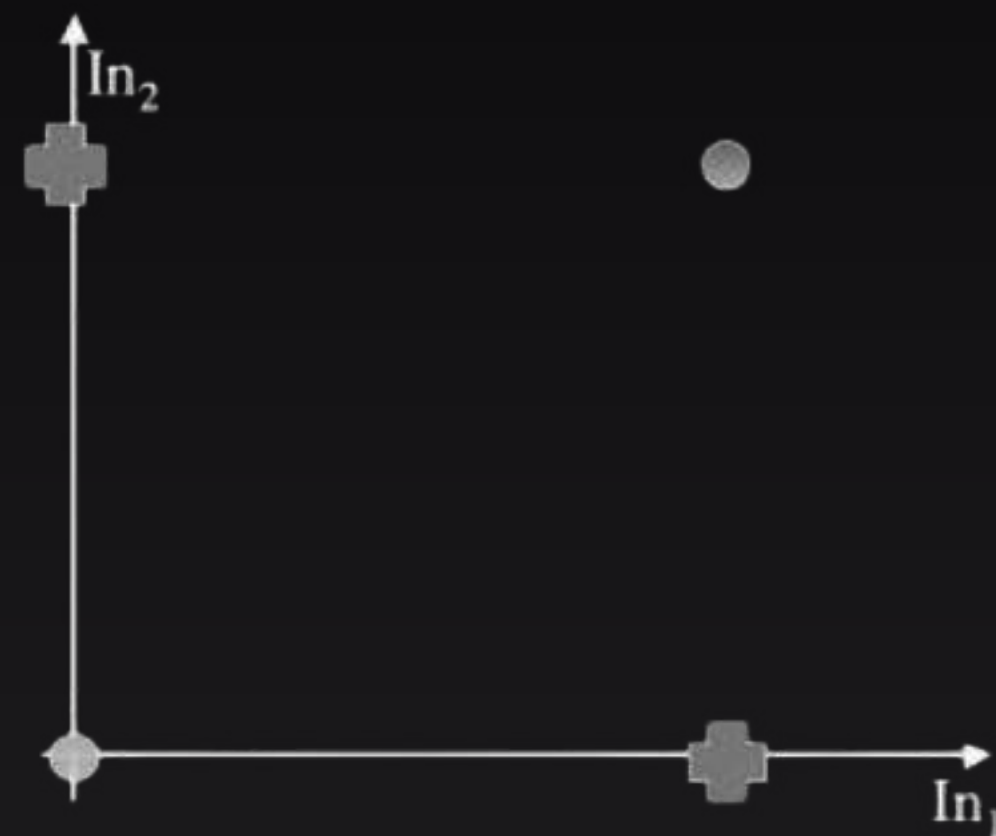
In_1	In_2	t
0	0	0
0	1	1
1	0	1
1	1	1



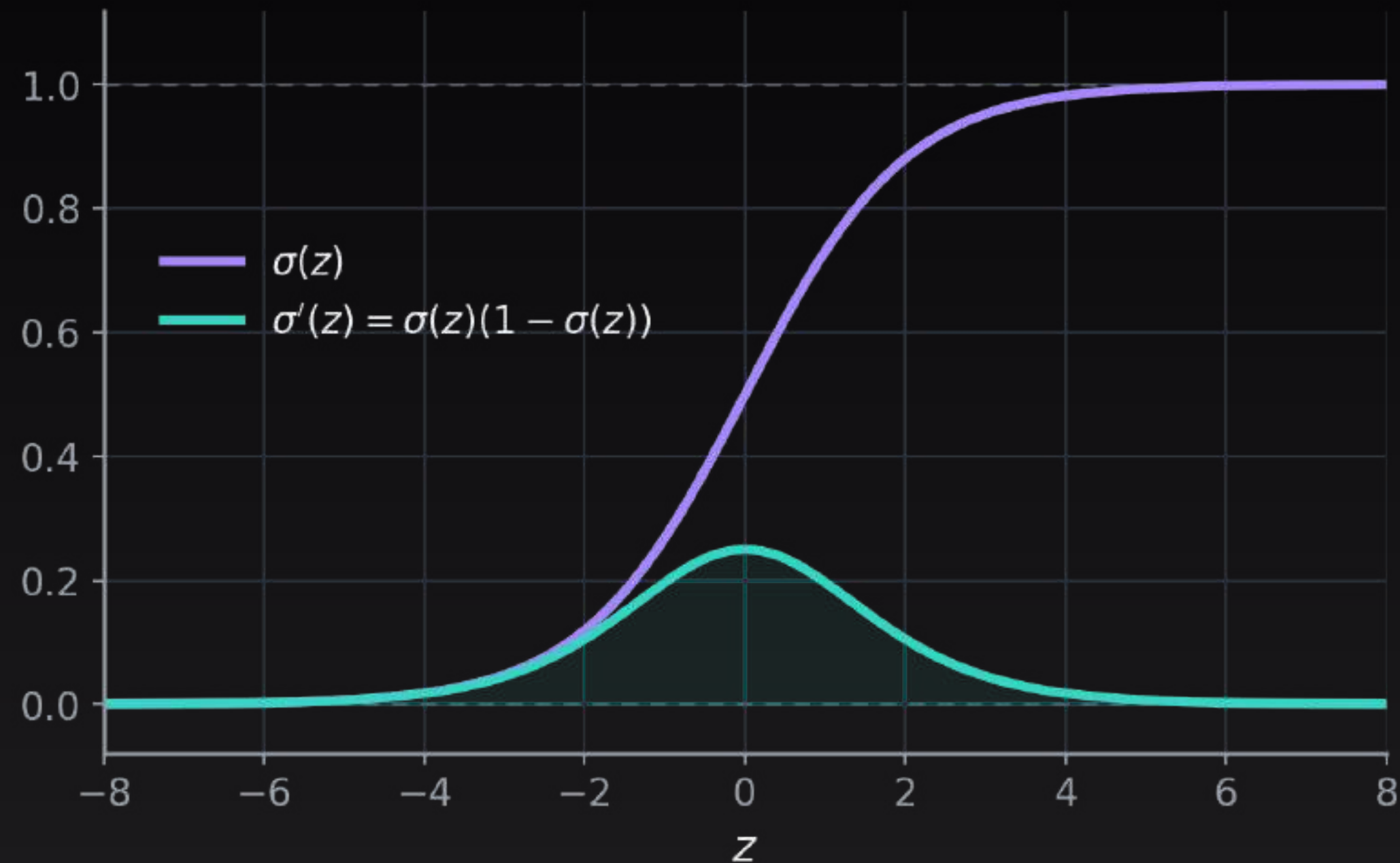
Why non-linearity?

The XOR Problem

In_1	In_2	t
0	0	0
0	1	1
1	0	1
1	1	0



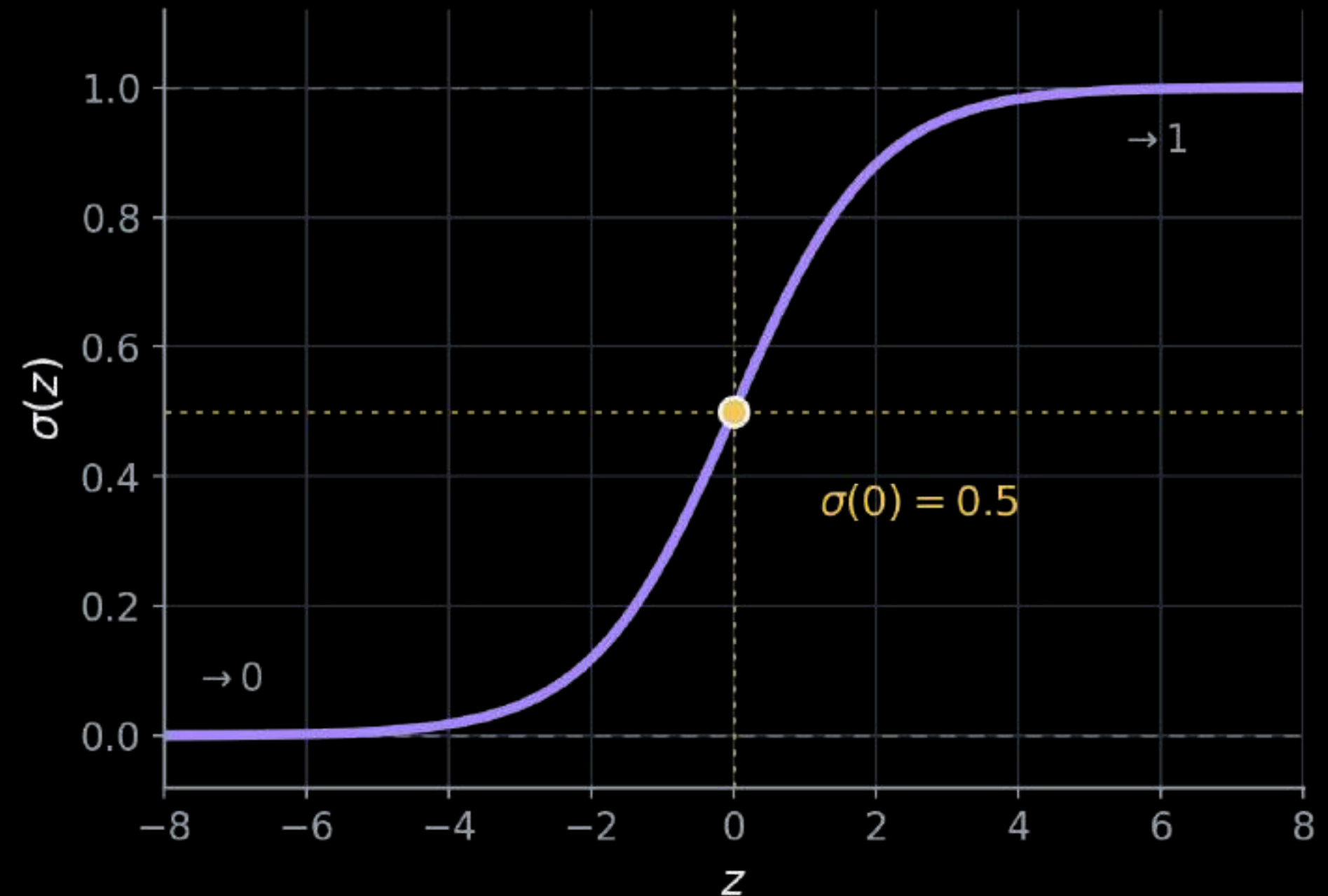
Sigmoid function and its gradient



Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Range (0, 1). Converts any real number into something resembling a probability.
- It is centered at $\frac{1}{2}$, passes through $\sigma(0) = 0.5$, and is monotonically increasing.
- Smooth and differentiable. Unlike the perceptron step function, it has no jumps: it is differentiable at every point.



Its “S”-shaped curve is flattened at the ends and is almost straight near $z = 0$.

Two mathematical tools for deriving it

The sigmoid function is a composition of functions, so we need:

Chain Rule

$$\frac{dg(f(x))}{dx} = \frac{dg}{df} \cdot \frac{df}{dx}$$

Power Rule

$$\frac{d[f(x)]^n}{dx} = n[f(x)]^{n-1} \cdot \frac{df(x)}{dx}$$

$$\frac{d}{dz} e^{-z} = -e^{-z}$$

Notice that the derivative of e^{-z} is again e^{-z} , but with a negative sign due to the chain rule (the partial derivative of $-z$ is -1).


$$\sigma(z) = (1 + e^{-z})^{-1}$$

$$\frac{d\sigma}{dz} = -(1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z})$$

$$= -(1 + e^{-z})^{-2} \cdot (-e^{-z})$$

$$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

$$\frac{e^{-z}}{1 + e^{-z}} = 1 - \frac{1}{1 + e^{-z}} = 1 - \sigma(z)$$

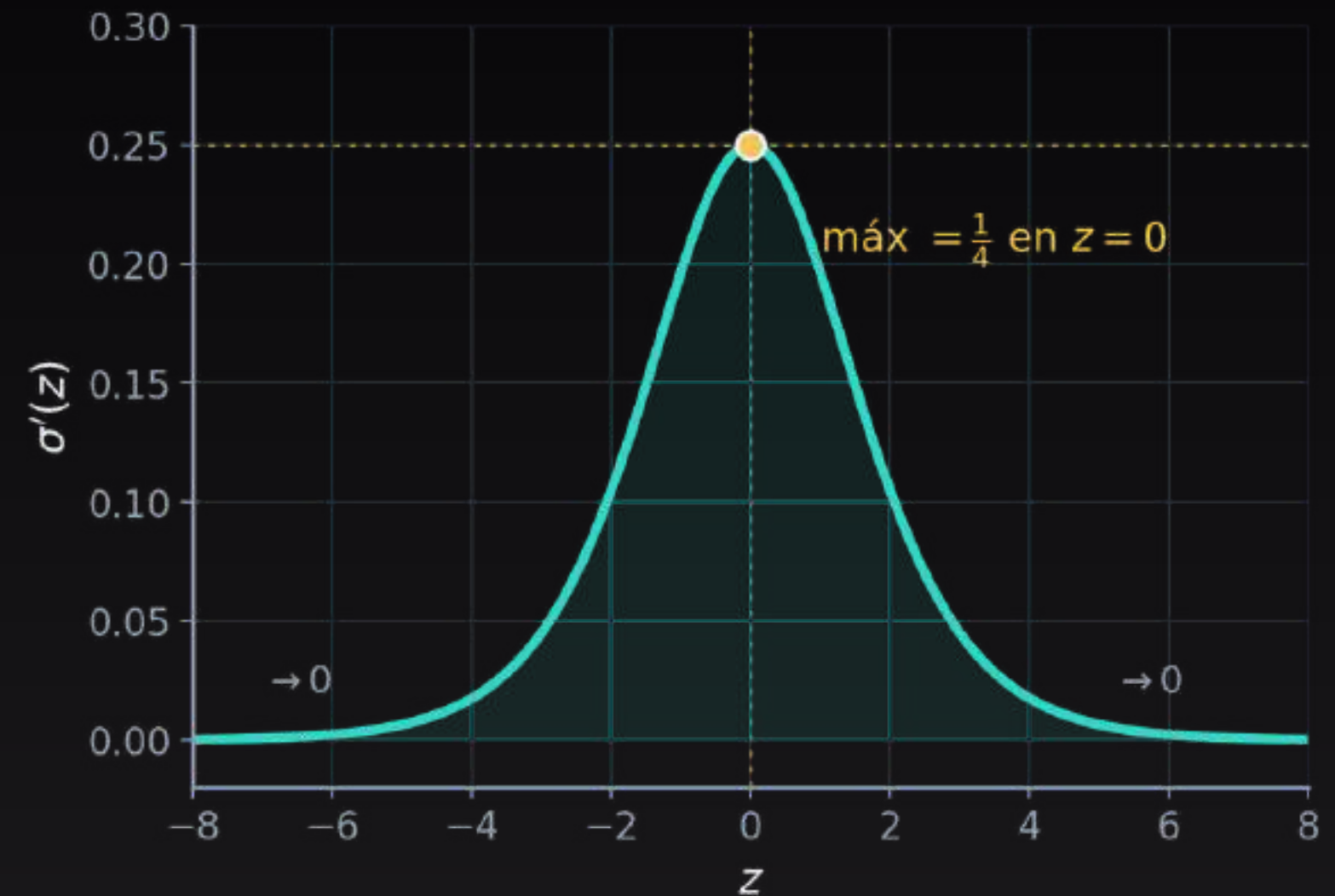
$$\frac{d\sigma}{dz} = \sigma(z) (1 - \sigma(z))$$

The gradient and an important consequence

- Maximum of $\frac{1}{4}$ at $z = 0$. Learning is faster where the sigmoid curve is steeper.
- Symmetric and always positive. It is bell-shaped and centered at zero.
- It tends to 0 at the extremes. When $|z|$ is large, the slope almost disappears.

⚠ FADING GRADIENTS

Since $\sigma'(z) \approx 0$ when the neuron is saturated, in very deep networks the corrections almost cancel each other out. That's why ReLU is now preferred in hidden layers.



ReLU and Its Gradient



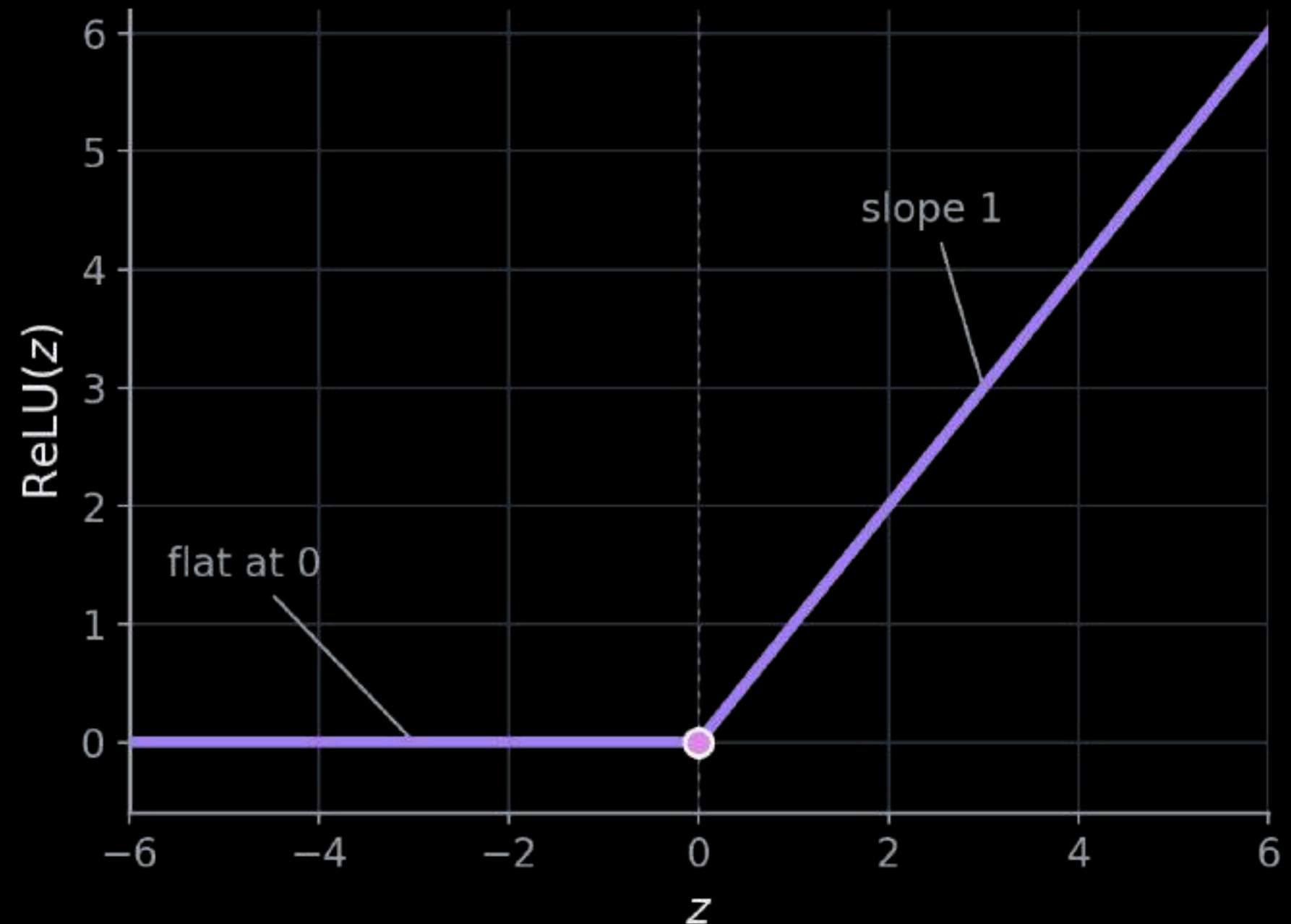
The rectified linear unit: a simple and fast activation function that avoids the fading gradient.

ReLU function

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}(z) = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

- Pass or block. Lets z through if positive, sends it to 0 if negative.
- **Cheap to compute.** Just a max – ideal for large networks.
- **No saturation on the right.** Grows without bound, unlike the sigmoid.



Flat on the left, a straight line with slope 1 on the right.

The gradient of ReLU

$$\text{ReLU}'(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

- **It is 1 or 0.** Either passes the full gradient through, or blocks it entirely.
- **No shrinking.** Unlike the sigmoid's $\leq 1/4$, here the factor is 1.



A step: ReLU's slope is 0 on the left and 1 on the right.

Strengths and a pitfall: the “dying ReLU”

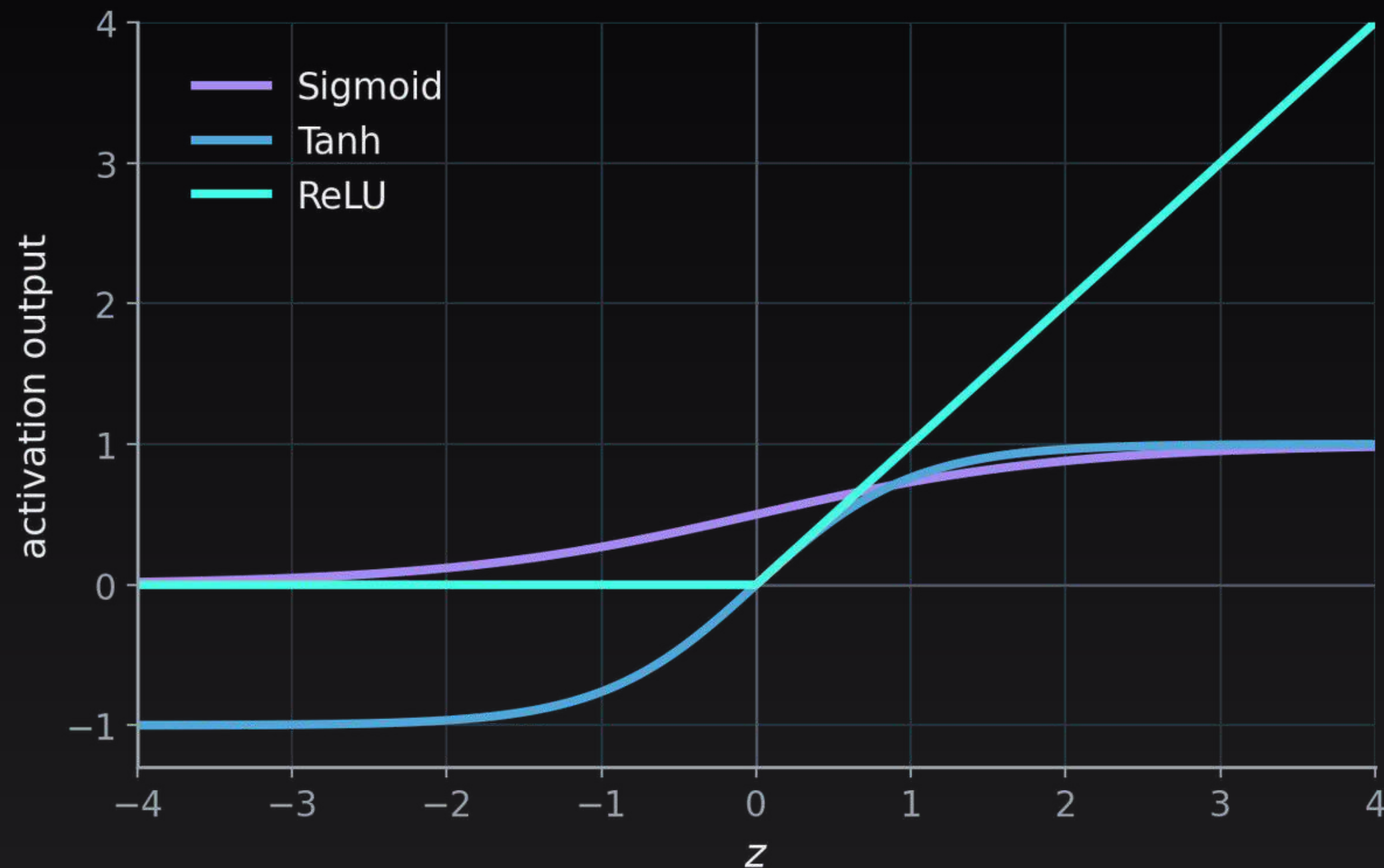
STRENGTHS

- **Simple and fast.** Just a $\max(0, z)$, no exponentials.
- **No vanishing.** The gradient is 1 in the active region.
- **No right-side saturation.** Active neurons keep learning.
- **Encourages sparsity.** Many outputs become 0 $\rightarrow$ more efficient representations.

⚠ The “Dying ReLU”

- If a neuron always receives $z < 0$, its gradient is 0 and its weights **never update**: it stays “dead” and stops learning forever.
- Fix: variants like Leaky ReLU or ELU, which give a small slope for $z < 0$.

Other Activation Functions and How to Pick One



A short comparison

- **Sigmoid** *range* $(0, 1)$

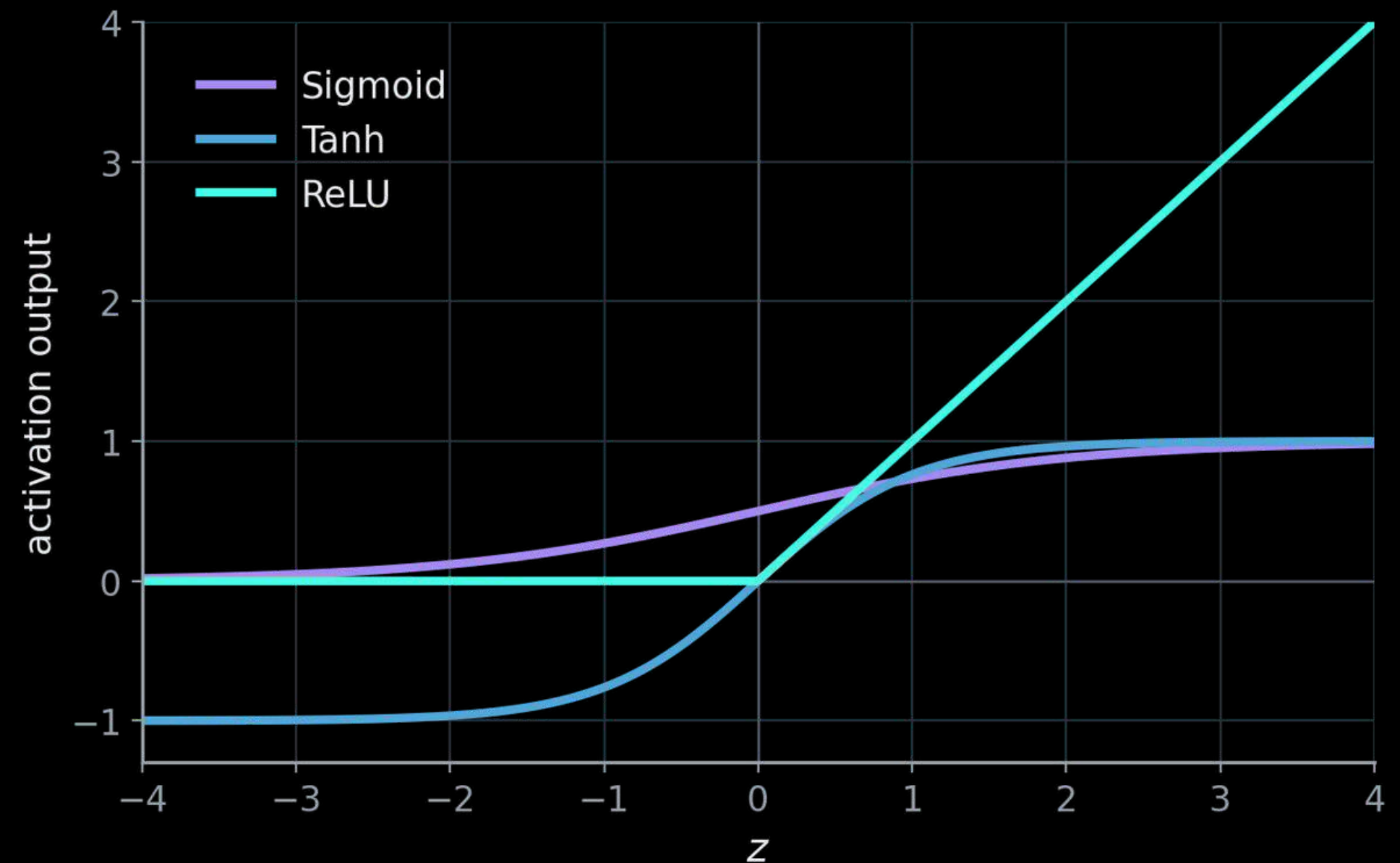
Squashes to a probability, but saturates
→ vanishing gradients.

- **Tanh** *range* $(-1, 1)$

Zero-centered (better than sigmoid), yet
still saturates.

- **ReLU** *range* $[0, \infty)$

No positive saturation, fast – the default
for hidden layers.

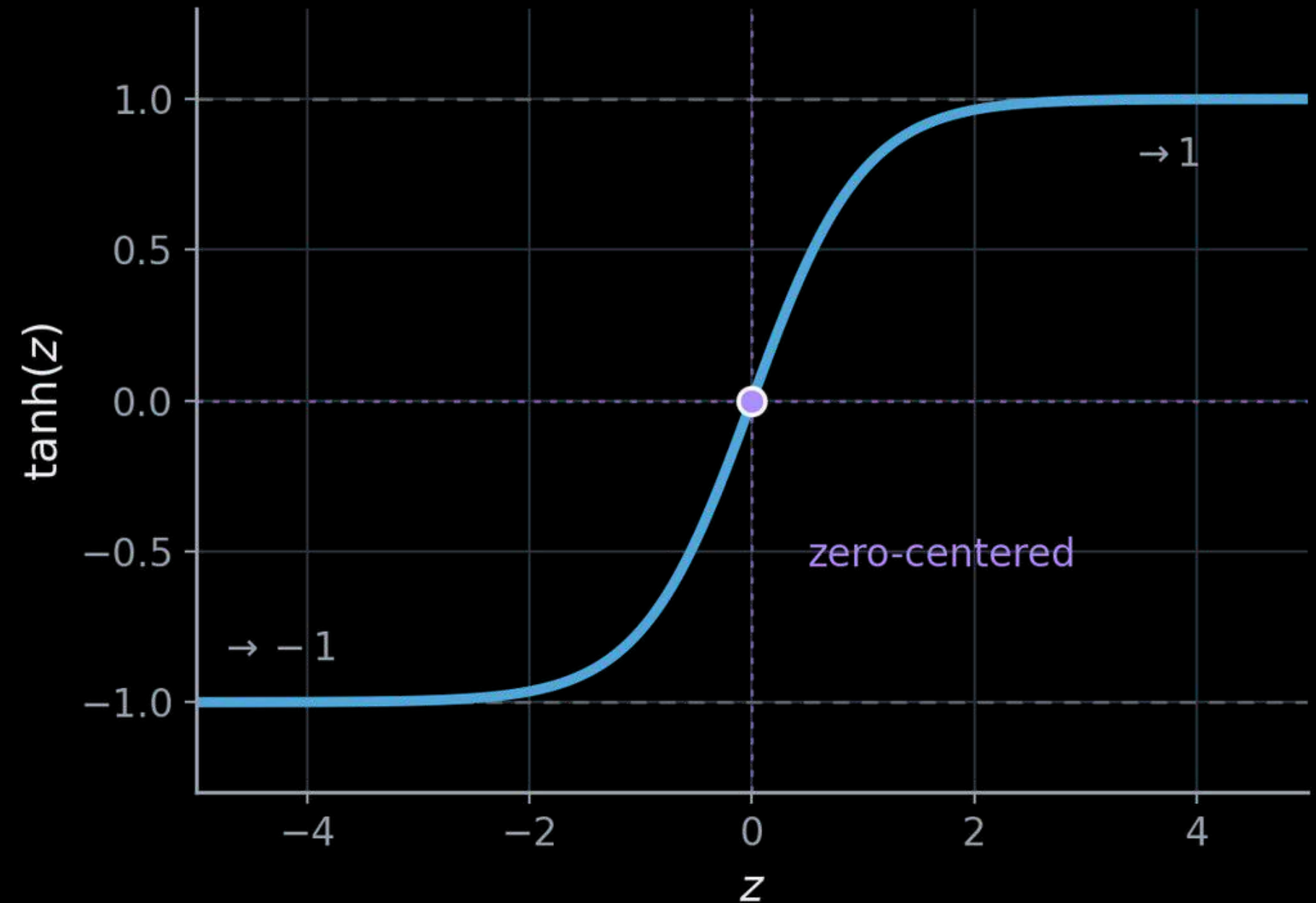


Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh(z) = 2 \sigma(2z) - 1$$

- **Range (-1, 1).** outputs are centered on 0, so gradients are better balanced than the sigmoid's.
- **Stronger gradients.** its slope near 0 is steeper, so learning starts faster.
- **Still saturates.** for large $|z|$ it flattens, so deep nets can still vanish – ReLU avoids this.

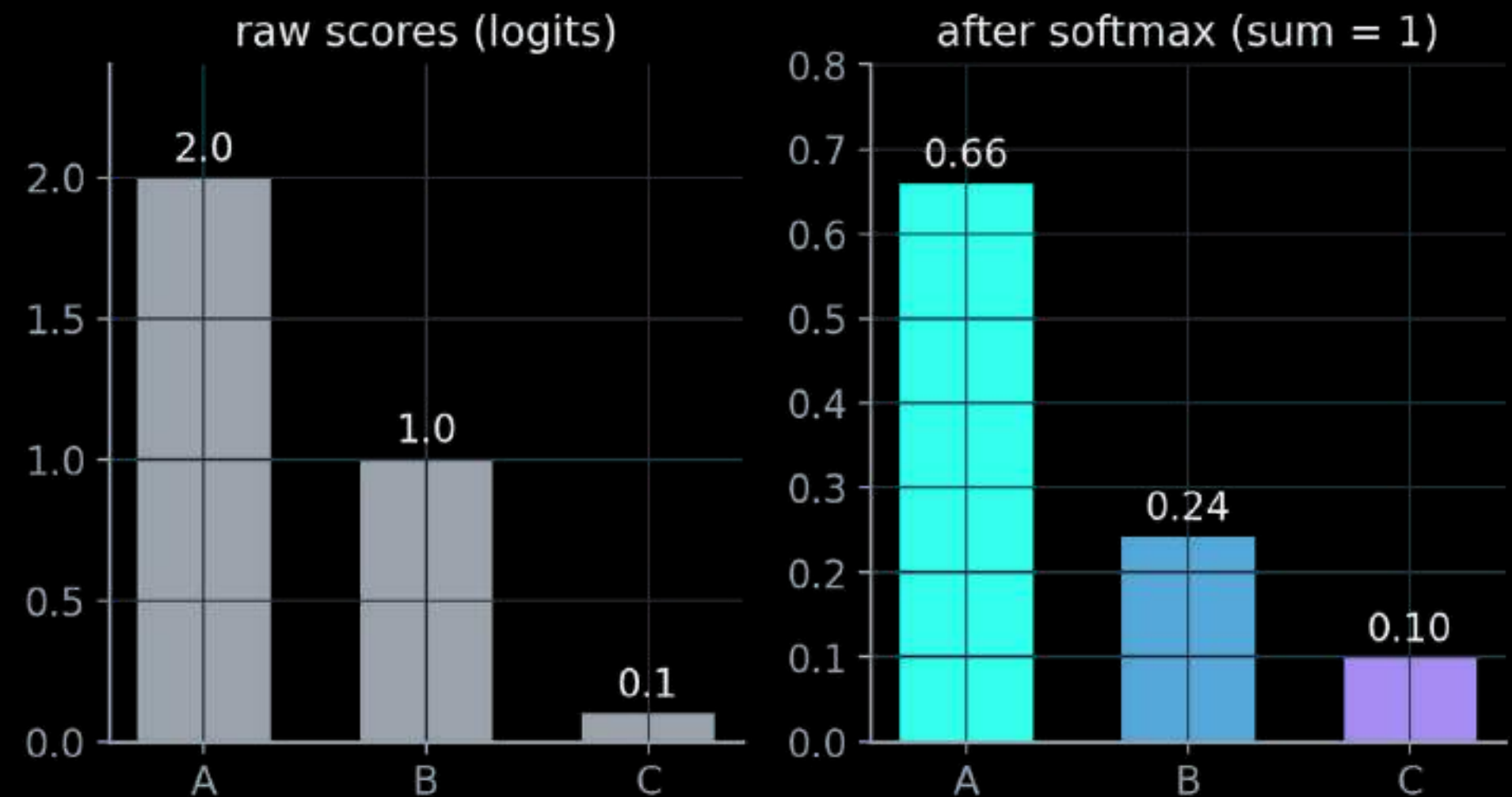


Same S-shape as the sigmoid, but it spans -1 to 1 and passes through the origin.

Softmax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- **A whole-vector activation.** unlike the others, it looks at all the scores together, not one at a time.
- **Outputs a distribution.** every value lands in (0, 1) and they all sum to exactly 1.
- **For multi-class output.** use it in the final layer when each input belongs to one of several classes.



Same S-shape as the sigmoid, but it spans -1 to 1 and passes through the origin.

The ReLU family

- ELU

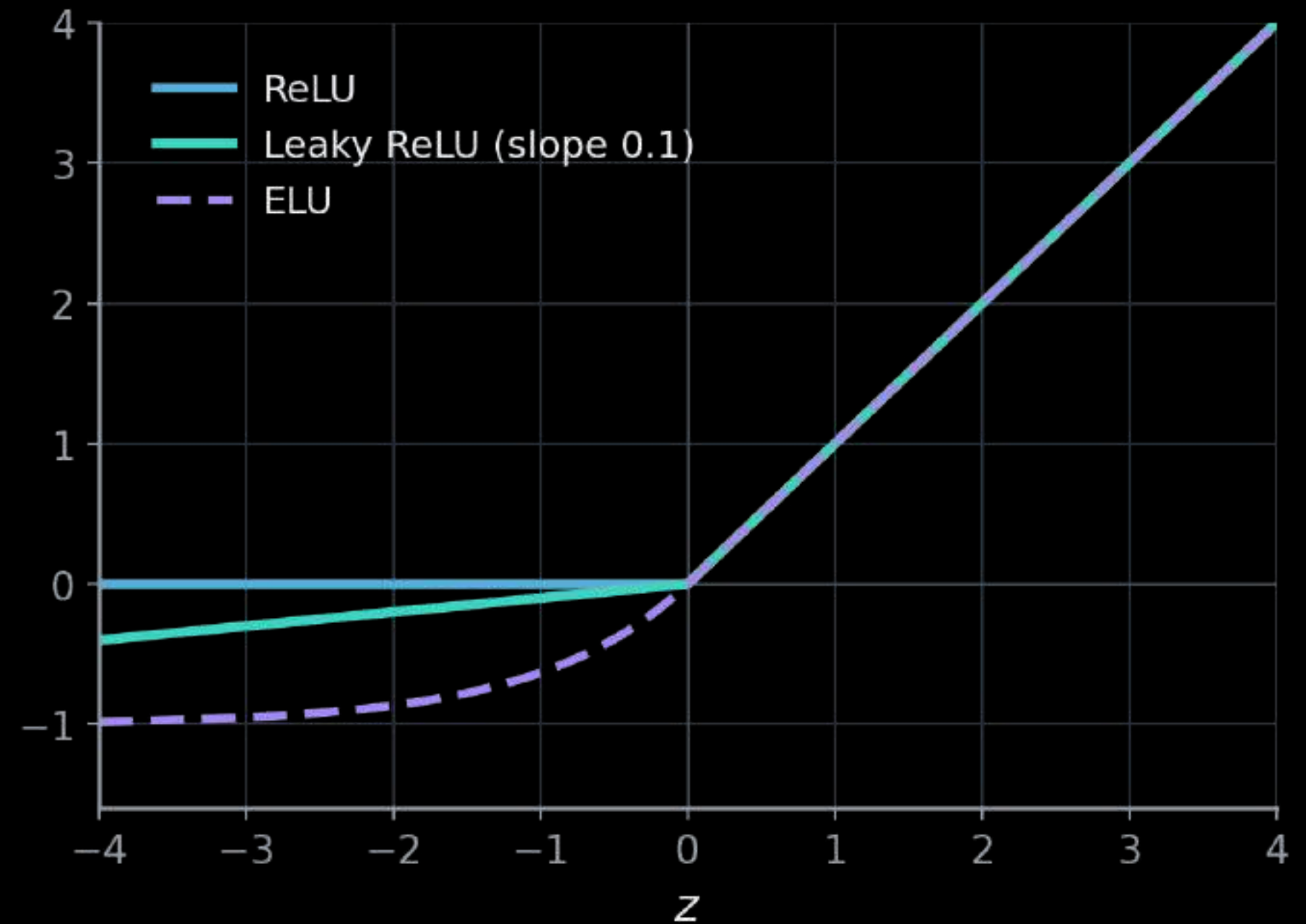
Smooth curve for negatives; outputs closer to zero-centered.

- GELU / others

Smooth, probabilistic gates used in modern transformers.

- Leaky ReLU

A small fixed slope (e.g. 0.1) for negatives – never fully dies.

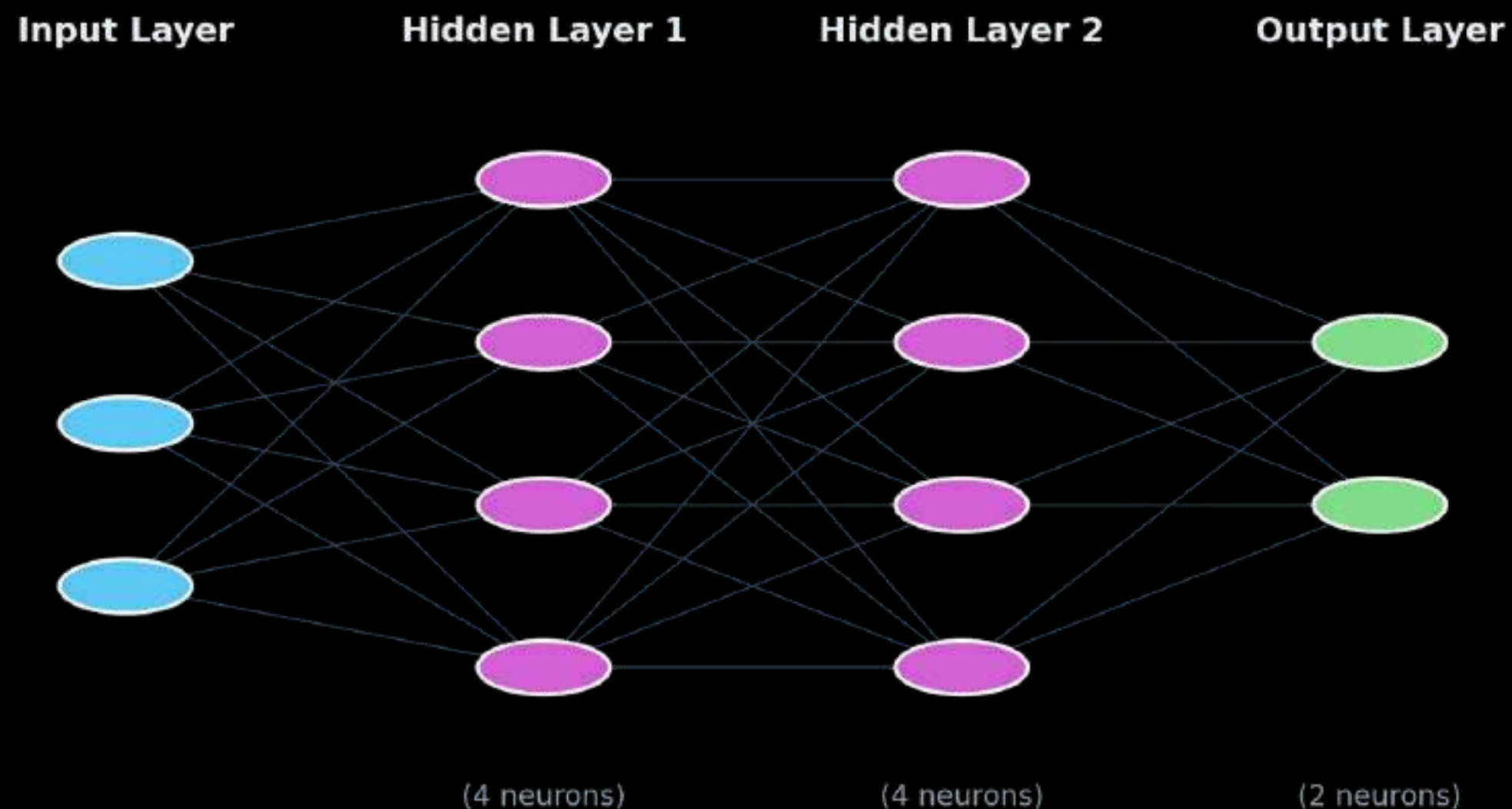


Same S-shape as the sigmoid, but it spans -1 to 1 and passes through the origin.

How to choose one?

Location	Recommended activation	Why
Hidden layers	ReLU (or variants)	Avoids vanishing gradients; cheap
Binary output	Sigmoid	Outputs a probability in (0, 1)
Multi-class output	Softmax	Outputs probabilities that sum to 1
Regression output	None (linear)	Can output any real value

The Role of Layers and Forward Propagation



- **Input**

Holds the raw features – one node per feature.

- **Hidden**

Do the real computation and learn representations.

- **Output**

Produce the final prediction.

Depth vs. width

Depth = number of layers

- “Deep” learning = many layers
- Enables hierarchical feature learning
- A 2-layer net has 2 weight matrices

Width = neurons per layer

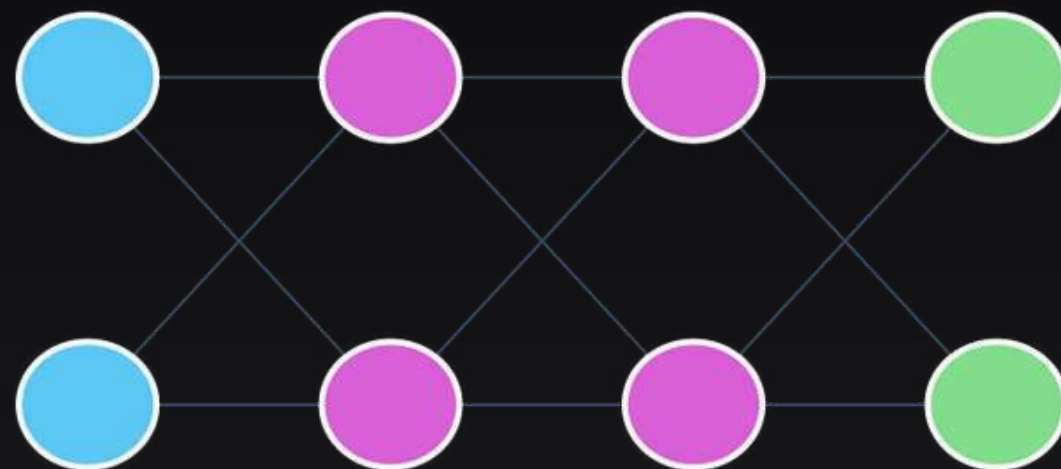
- More neurons = more capacity per layer
- Can be “wide” without being “deep”
- A 2-layer net has 2 weight matrices

Depth vs. width



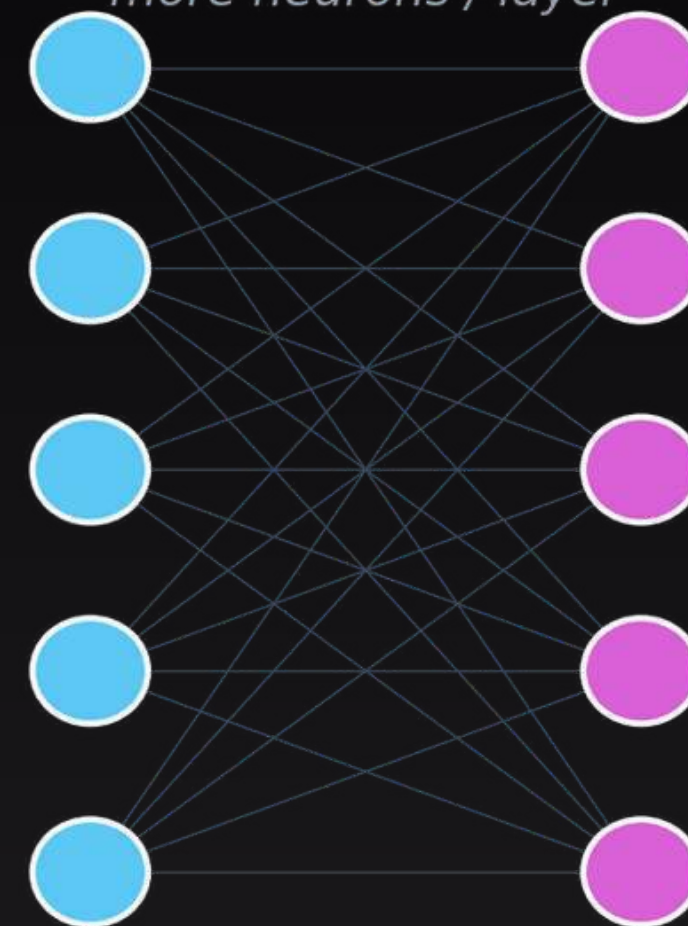
Deep

more layers

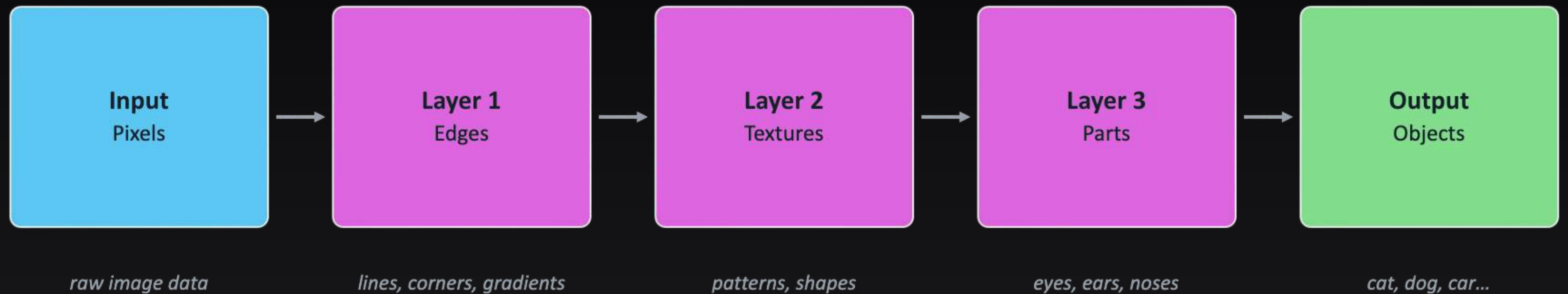


Wide

more neurons / layer



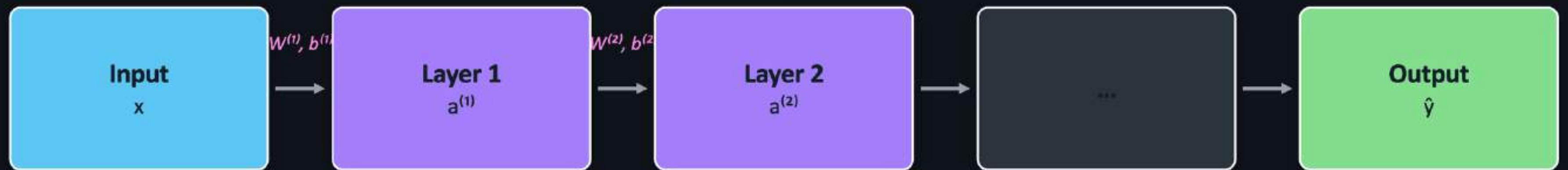
Why Add More Layers? Understanding Depth



Depth lets the network compose simple features into meaningful concepts – something a single wide layer cannot do.



Why Add More Layers? Understanding Depth



At each layer:

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \text{Preactivation}$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) \quad \text{Apply the activation}$$



Illustrative Example

Input

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \text{ (2 features)}$$

Layer 1 (2 neurons)

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{bmatrix}, \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$$

Layer 2 (1 neuron, output)

$$W^{(2)} = \begin{bmatrix} 0.6 & 0.8 \end{bmatrix}, b^{(2)} = 0.2$$

Pre-activation

$$\begin{aligned}\mathbf{z}^{(1)} &= W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \\ &= \begin{bmatrix} 0.5(1) + (-0.3)(2) \\ 0.2(1) + 0.4(2) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \\ &= \begin{bmatrix} -0.1 \\ 1.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.9 \end{bmatrix}\end{aligned}$$

Activation

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \text{ReLU} \begin{bmatrix} 0 \\ 0.9 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.9 \end{bmatrix}$$

Layer 2 takes $\mathbf{a}^{(1)} = [0, 0.9]$ and produces the final value:

$$z^{(2)} = W^{(2)} \mathbf{a}^{(1)} + b^{(2)} = \begin{bmatrix} 0.6 & 0.8 \end{bmatrix} \begin{bmatrix} 0 \\ 0.9 \end{bmatrix} + 0.2$$

$$= 0.6(0) + 0.8(0.9) + 0.2 = 0.92$$

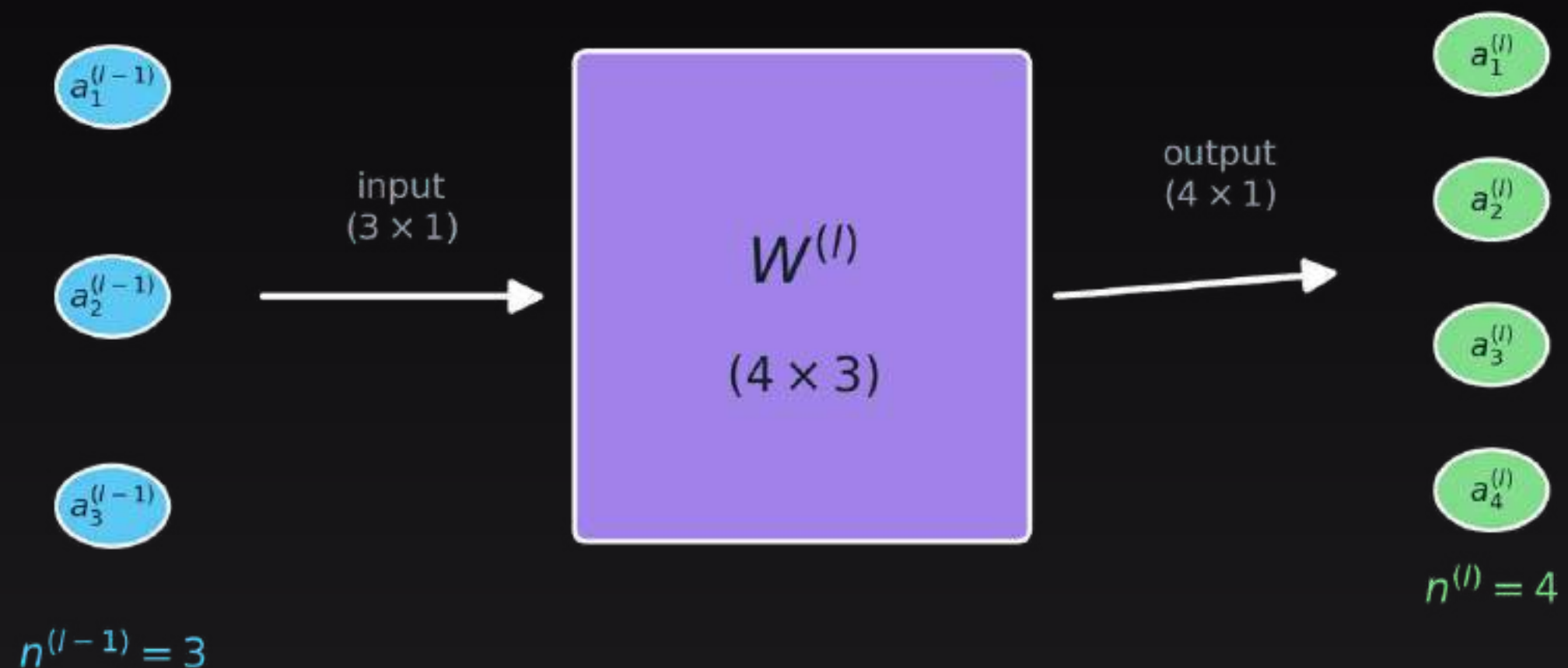
$$\hat{y} = z^{(2)} = 0.92$$

The output activation depends on the task: **linear** $\rightarrow 0.92$, **sigmoid** $\rightarrow \sigma(0.92) \approx 0.72$.

That is forward propagation!: a chain of $z = Wa + b$ followed by an activation, layer after layer.

Matrix notation & dimensions

Packing a layer's math into matrices, keeping every **shape** consistent, and counting what the network actually **learns**.



A whole layer in two matrix lines

For any layer ℓ , the connection to the previous layer is fully captured by:

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$$

$$\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$$

- Input

As many as the neurons feeding in – that count is $n^{(\ell-1)}$.

- Outputs.

One per neuron in this layer – that count is $n^{(\ell)}$.

- Same form everywhere

Every layer obeys these two identical lines.

- Two steps.

One per neuron in this layer – that count is $n^{(\ell)}$.

Getting every shape right

Symbol	Shape	What it is
$W^{(l)}$	$n^{(l)} \times n^{(l-1)}$	the layer's weights
$b^{(l)}$	$n^{(l)} \times 1$	the layer's biases
$a^{(l-1)}$	$n^{(l-1)} \times 1$	what comes in
$a^{(l)}$	$n^{(l)} \times 1$	what goes out

Making the shapes line up

Each operation only works if its dimensions agree. Track them symbol by symbol:

In general

$$(n^{(l)} \times n^{(l-1)}) \cdot (n^{(l-1)} \times 1) + (n^{(l)} \times 1) = (n^{(l)} \times 1)$$

With the numbers from the diagram

$$(4 \times 3) \cdot (3 \times 1) + (4 \times 1) = (4 \times 1)$$

The inner dimensions must match!

$$(4 \times 3) \times (3 \times 1) = (4 \times 1)$$

Running many examples in one shot

One example at a time. $a^{(l-1)}$ is a single column, shape $(n^{(l-1)} \times 1)$.

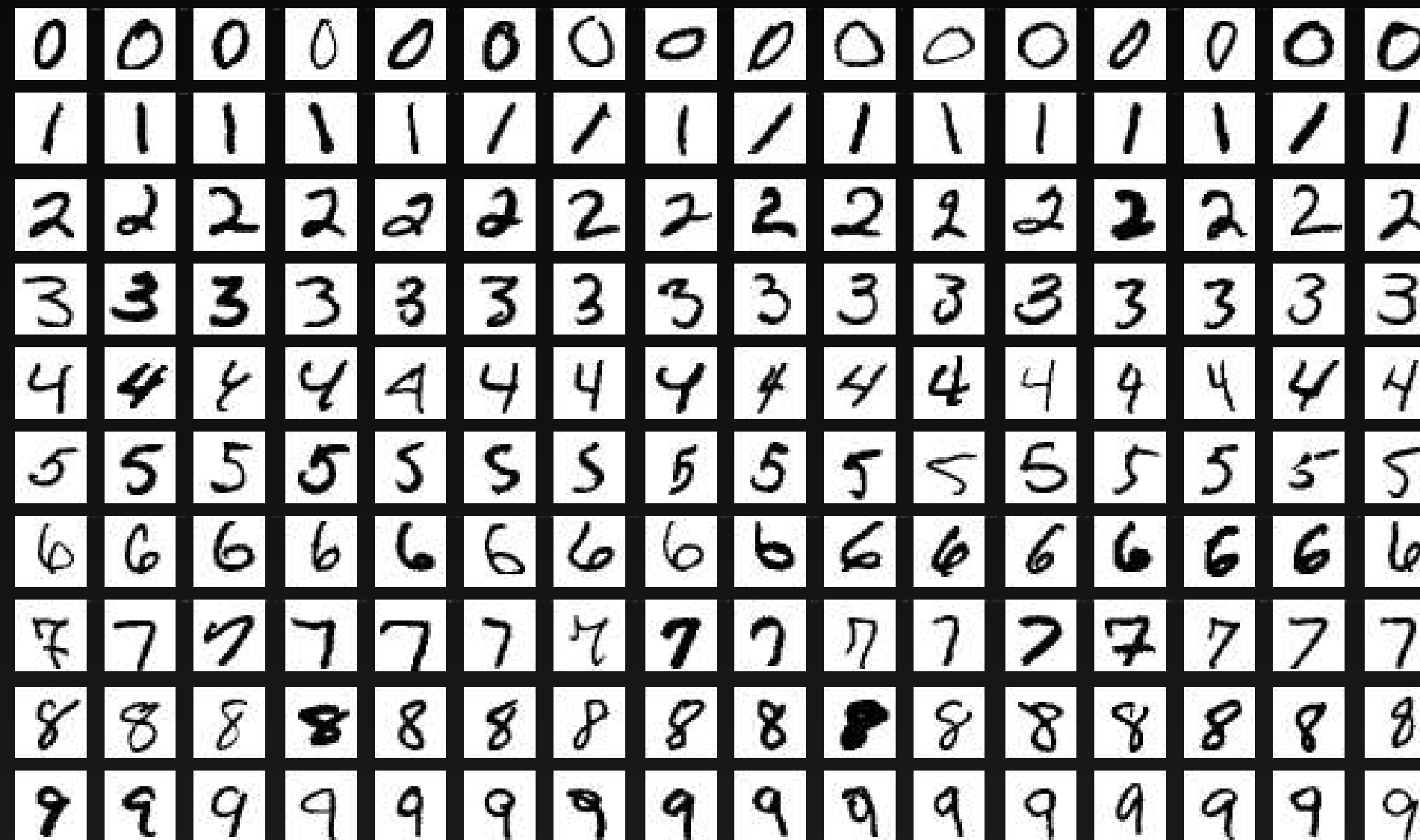
A batch of B examples: Stack them side by side $\rightarrow A^{(l-1)}$ becomes $(n^{(l-1)} \times B)$.

$$Z^{(l)} = W^{(l)} A^{(l-1)} + \mathbf{b}^{(l)}$$

$$(n^{(l)} \times n^{(l-1)}) \cdot (n^{(l-1)} \times B) = (n^{(l)} \times B)$$

The payoff: the entire batch flows through in a single matrix multiply – exactly what GPUs love.

A worked case: a hand written-digit classifier (MINIST)



Architecture: 784 → 128 → 10

RESULTING WEIGHT SHAPES

- **Input:** each 28×28 image flattens to 784 numbers.
- **Hidden:** 128 neurons with a ReLU activation.
- **Output:** 10 neurons (one per digit) with softmax.

$$W^{(1)} \quad 128 \times 784$$

$$b^{(1)} \quad 128 \times 1$$

$$W^{(2)} \quad 10 \times 128$$

$$b^{(2)} \quad 10 \times 1$$

Read each as (outputs × inputs) – the rule from before.

How many numbers does it learn?

Total parameters = every weight plus every bias, added up across the network.

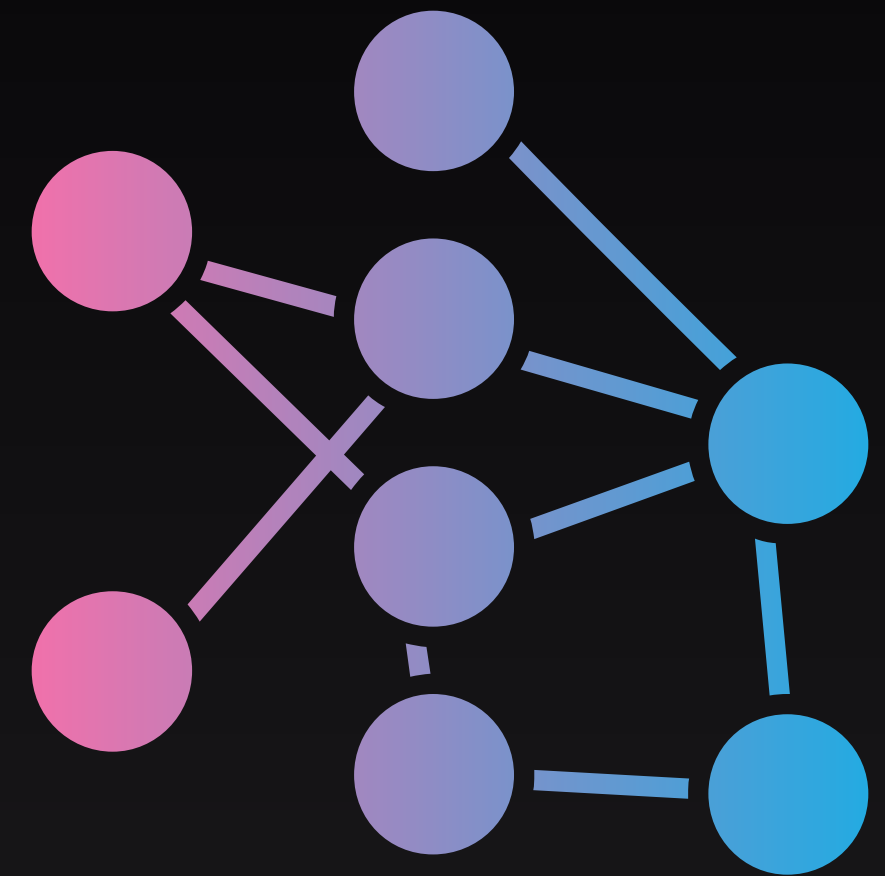
Layer 1

- **Weights:** $128 \times 784 = 100,352$
- **Biases:** 128

Layer 2

- **Weights:** $10 \times 128 = 1,280$
- **Biases:** 10

Total: $100,352 + 128 + 1,280 + 10 = 101,770$ parameters



The same count, as one formula

For a network whose layer sizes are:

$$n^{(0)} \rightarrow n^{(1)} \rightarrow n^{(2)} \rightarrow \dots \rightarrow n^{(L)}$$

In each layer ℓ :

<i>Weights:</i>	$n^{(\ell)} \times n^{(\ell-1)}$
<i>Biases:</i>	$n^{(\ell)}$
<i>Per layer:</i>	$n^{(\ell)} \times (n^{(\ell-1)} + 1)$

Add it across all layers:

$$\sum_{\ell=1}^L n^{(\ell)} \times (n^{(\ell-1)} + 1)$$



Why we lean on matrices at all

Clearer to reason about

- **Compact:** a layer is one line, not a loop over neurons.
- **Traceable:** you can follow the data's shape end to end.
- **Self-checking:** mismatched dimensions reveal bugs instantly.

Faster to compute

- **Batchable:** many samples ride through together.
- **GPU-friendly:** hardware is built for big matrix multiplies.
- **Parallel:** all samples and neurons compute at once.

Strip away the diagrams and deep learning is, at heart, a stack of matrix multiplications.